



Continuous Delivery (CD) Workflow

Introduction

Continuous Delivery (CD) is a **DevOps practice** that ensures code is **always in a deployable state**.

- What is Continuous Delivery?
- How does it differ from Continuous Integration (CI)?
- How can you implement a Continuous Delivery workflow in Jenkins?

This guide provides an **in-depth understanding of the Continuous Delivery workflow**.

Section 1: Learn

What is Continuous Delivery (CD)?

CD is the process of **automating software release and deployment to staging or production environments**.

Difference Between CI and CD

Feature	Continuous Integration (CI)	Continuous Delivery (CD)
Focus	Merging and testing code	Releasing software
Automation Level	Builds and tests automatically	Prepares deployment automatically
Manual Steps	None (fully automated)	Deployment approval may be required



Feature	Continuous Integration (CI)	Continuous Delivery (CD)
Goal	Catch errors early	Deliver deployable software anytime
Tools Used	Jenkins, GitHub Actions, Travis CI	Jenkins, Spinnaker, AWS CodeDeploy

Why Use Continuous Delivery?

Benefit	Description
Faster Releases	Speeds up software deployment
Fewer Errors	Automates testing and validation
Rollback Support	Allows easy rollback to previous versions
Scalability	Supports large-scale deployments
Works with Cloud	Can deploy to AWS, Azure, Kubernetes

Interesting Fact

Companies like **Google, Amazon, and Netflix** deploy **hundreds of updates per day** using CD pipelines.



Section 2: Practice

1. Setting Up a CD Workflow in Jenkins

Step 1: Install Required Plugins

1. Click **Manage Jenkins** → **Manage Plugins**.
2. Install:
 - **Pipeline Plugin** (for CD automation)
 - **SSH Plugin** (for remote deployments)
 - **AWS CodeDeploy Plugin** (for cloud deployment)

Step 2: Create a CD Pipeline Job in Jenkins

1. Click **New Item** → Select **Pipeline**.
2. Scroll to **Pipeline Definition** → Select **Pipeline script from SCM**.
3. Enter the **Git Repository URL** where the Jenkinsfile is stored.

Step 3: Define the Jenkinsfile for CD Workflow

Create a file named **Jenkinsfile** in your **Git repository** with the following script:

```
pipeline {  
  agent any  
  stages {  
    stage('Checkout') {  
      steps {  
        git 'https://github.com/your-repo.git'  
      }  
    }  
    stage('Build') {  
      steps {
```



```
    sh 'mvn clean install'
  }
}
stage('Test') {
  steps {
    sh 'mvn test'
  }
}
stage('Package') {
  steps {
    sh 'mvn package'
  }
}
stage('Deploy') {
  steps {
    echo 'Deploying application to staging environment...'
  }
}
}
```

Step 4: Save and Run the CD Pipeline

1. Click **Save** → **Build Now**.
2. Check the **Console Output** for the deployment log.

Why is this useful?

- Ensures **every code change can be deployed easily**.



2. Implementing Manual Approval for Production Deployment

Step 1: Add an Input Step for Approval

Modify the **Jenkinsfile** to include a **manual approval step** before deploying to production:

```
pipeline {
  agent any
  stages {
    stage('Deploy to Staging') {
      steps {
        echo 'Deploying to Staging...'
      }
    }
    stage('Approval Required') {
      steps {
        input "Proceed to Production Deployment?"
      }
    }
    stage('Deploy to Production') {
      steps {
        echo 'Deploying to Production...'
      }
    }
  }
}
```



```
}
```

Step 2: Run the Pipeline

1. Jenkins will **pause the execution** at the **Approval Required** stage.
2. A user must **manually approve** the deployment.
3. Once approved, Jenkins proceeds to **deploy the application to production**.

Why is this useful?

- Adds **human verification** before releasing software to customers.
-

3. Automating CD with AWS CodeDeploy

CD can be extended to **deploy applications to AWS automatically**.

Step 1: Install AWS CLI and Configure Jenkins

1. Install AWS CLI on the Jenkins server:

```
sudo apt install awscli -y
```

2. Configure AWS credentials:

```
aws configure
```

Step 2: Modify Jenkinsfile for AWS Deployment

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {
```



```
    sh 'mvn clean install'
  }
}
stage('Deploy to AWS') {
  steps {
    sh 'aws deploy push --application-name MyApp --s3-location
s3://my-bucket/deployment.zip'
  }
}
}
```

Why is this useful?

- **Automates cloud deployment**, reducing manual work.
-

Section 3: Know More

Frequently Asked Questions

1. **What is the goal of Continuous Delivery?**
 - To ensure **code is always in a deployable state**.
2. **How does CD differ from CI?**
 - CI focuses on **testing code**, while CD ensures **it's ready for deployment**.
3. **What happens if a CD build fails?**
 - Jenkins logs errors and **alerts developers to fix issues before deployment**.
4. **Can CD be fully automated?**



- Yes, but **manual approval can be added for production releases.**
5. **What cloud platforms support CD?**
- AWS, Azure, Google Cloud, Kubernetes, Docker Swarm.
-

Conclusion

Continuous Delivery **automates software deployment, reducing release cycles and improving reliability.**

Start by **creating a Jenkins CD pipeline, adding approval steps, and integrating cloud deployment,** and soon you'll be **deploying high-quality software efficiently!**